Practical 7

Aim: Model a simple Bayesian network or probabilistic classifier for prediction or diagnosis and perform inference using a suitable Python library.

```
# Function to predict health risk def
analyze_health_risk(evidence):     #
Start with a basic risk score      risk
= 20

   # Increase risk based on the given information
if evidence["age_group"] == "40-60":
      risk += 10

   if evidence["smoking"] == "Yes":
      risk += 15

   if evidence["exercise"] == "Low":
      risk += 10

   if evidence["blood_pressure"] == "High":
      risk += 15

   if evidence["cholesterol"] == "High":
      risk += 10

   if evidence["family_history"] == "Yes":
      risk += 10

   if evidence["chest_pain"] == "Yes":
      risk += 10

   # Keep the risk between 0 and 100
risk = min(risk, 100)

   # Calculate low risk percentage
low_risk = 100 - risk

   # Return the results
return risk, low_risk # Main
program def main():
```

```
    print("HEALTH RISK PREDICTION SYSTEM")

    # Take sample health information
evidence = {        "age_group":
"40-60",
        "smoking": "Yes",
        "exercise": "Low",
        "blood_pressure": "High",
        "cholesterol": "High",
        "family_history": "Yes",
        "chest_pain": "No"
    }

    # Display the input information
print("\nInput Information:")     for
key, value in evidence.items():
        print(key, ":", value)

    # Predict the risk     high_risk, low_risk =
analyze_health_risk(evidence)

    # Display the result
print("\nPrediction Result:")
print("High Risk:", high_risk, "%")
print("Low Risk :", low_risk, "%")

    # Display a simple conclusion
if high_risk >= 50:
        print("Result: Higher estimated risk")
else:
        print("Result: Lower estimated risk")

    print("\nNote: This is only an educational demonstration.")


# Start the program if
__name__ == "__main__":
main()
```

OUTPUT

```
HEALTH RISK PREDICTION SYSTEM

Input Information:
age_group : 40-60
smoking : Yes
exercise : Low
blood_pressure : High
cholesterol : High
family_history : Yes
chest_pain : No

Prediction Result:
High Risk: 90 %
Low Risk : 10 %
Result: Higher estimated risk

Note: This is only an educational demonstration.
```

Practical 8

Aim: Develop a mini AI application such as text classification, chatbot prototype or perception demo and report on ethics and explainability considerations

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

# Training data tickets
= [
   "wifi is not working",
   "network connection failed",
   "printer is not printing",
   "printer shows error",
   "install python software",
   "install application",
   "server is down",
   "database server crashed",
   "forgot my password",
   "cannot login to account"
]

# Categories of tickets categories
= [
   "Network", "Network",
   "Printer", "Printer",
   "Software", "Software",
   "Server", "Server",
   "Account", "Account"
]

# Convert text into numerical values vectorizer
= TfidfVectorizer()
X = vectorizer.fit_transform(tickets)

# Train the AI model model =
LogisticRegression()
model.fit(X, categories)

# Enter a new ticket
```

```
ticket = "printer is not working"

# Predict its category new_ticket =
vectorizer.transform([ticket]) category =
model.predict(new_ticket)[0]

# Assign priority using simple rules if "server" in
ticket.lower() or "crashed" in ticket.lower():
   priority = "High" elif "error" in ticket.lower() or "not
working" in ticket.lower():
   priority = "Medium"
else:     priority =
"Low"

# Display the result
print("IT HELPDESK TICKET ANALYZER")
print("Ticket:", ticket) print("Category:",
category) print("Priority:", priority)
```

OUTPUT

```
IT HELPDESK TICKET ANALYZER
Ticket: printer is not working
Category: Network
Priority: Medium
```

Practical 9

Aim: Implement a simple planning problem using STRIPS / goal-stack planning for Blocks World, logistics, or robot navigation

```
# Initial state: The robot is at location A initial_state
= {"At_A"}

# Goal: The robot must reach location C goal
= "At_C"

# Define all available actions actions
= [
   {
      "name": "Move A to B",
      "pre": {"At_A"},       # Condition: Robot must be at A
      "add": {"At_B"},       # Result: Robot reaches B
      "delete": {"At_A"}     # Remove the old position A
   },
   {
      "name": "Move B to C",
      "pre": {"At_B"},       # Condition: Robot must be at B
      "add": {"At_C"},       # Result: Robot reaches C
      "delete": {"At_B"}     # Remove the old position B
   }
]


# Find an action that can achieve the given goal
def find_action(goal):     for action in actions:
      # Check whether this action produces the goal
if goal in action["add"]:
         return action

   # Return None if no action can achieve the goal
return None


# Create a plan to achieve the goal def
make_plan(goal, state, plan):
```

```
    # Step 1: Check if the goal is already achieved
if goal in state:         return True

    # Step 2: Find an action that achieves the goal
action = find_action(goal)

    # If no suitable action exists, planning fails
if action is None:         return False

    # Step 3: Achieve all preconditions first
for condition in action["pre"]:
        # Recursively make the required condition true
if not make_plan(condition, state, plan):
            return False

    # Step 4: Execute the action     #
Remove facts that are no longer true
state.difference_update(action["delete"])

    # Add new facts that become true
state.update(action["add"])

    # Store the action in the plan
plan.append(action["name"])

    return True


# Create a copy of the initial state state
= initial_state.copy()

# Create an empty list to store the plan plan
= []

# Run the planning algorithm success
= make_plan(goal, state, plan)

# Display the result if
success:
print("Plan:")
    for step in plan:
print(step)
```

```
print("Final State:", state)
print("Goal Achieved!") else:
print("Goal cannot be achieved.")
```

OUTPUT:

```
Plan:
Move A to B
Move B to C
Final State: {'At_C'}
Goal Achieved!
```